
From last lab session

Laboratorio Trattamento Numerico dei Dati Sperimentali

L. Carminati
Universita' degli studi di Milano

Summary sulle formule di quadratura di Newton-Cotes

Grado Polinomio	Nr. Punti	Errore	Grado	Errore composta	Formula	Tipo
0	1	$-\frac{(b-a)^3 f''(\xi_1)}{24}$	1	$\sim h^2$	Midpoint	Aperta
1	2	$-\frac{(b-a)^3 f''(\xi_2)}{12}$	1	$\sim h^2$	Trapezoidi	Chiusa
2	3	$-\frac{(b-a)^5 f^{IV}(\xi_3)}{2880}$	3	$\sim h^4$	Simpson	Chiusa

Implementazione proposta metodo Midpoint

- ❑ Il metodo proposto nel Progetto repl aveva due errori, nel caso non siate riusciti a trovarli:

```
double Integral::Midpoint(unsigned int nstep){  
    m_h = (m_b - m_a)/double(nstep);  
    m_sum = 0.;  
  
    for (unsigned int i=1; i<nstep; i++){  
        m_sum += m_f->Eval( m_a + (i+0.5)*m_h );  
    }  
  
    m_integral = m_sign*m_sum;  
  
    return m_integral;  
}
```

Il ciclo deve partire da 0 altrimenti escludiamo il punto medio del primo intervallo

Qui chiaramente manca la moltiplicazione per la base m_h !

Andamento dell'errore

```
#include "Integrale.h"
#include "Funzioni.h"

#include <cmath>
#include <iostream>
#include <iomanip>
#include <cstdlib>

#include "TGraph.h"
#include "TApplication.h"
#include "TAxis.h"
#include "TPad.h"
#include "TCanvas.h"
#include "TLegend.h"

using namespace std;

int main ( ) {

    TApplication app("app",0,0);

    Seno *f = new Seno();
    double a=0., b=M_PI, esatto=2. ;

    Simpson *mySimpson = new Simpson(a,b,f);
    Midpoint *myMidpoint = new Midpoint(a,b,f);

    TGraph gSimpson;
    TGraph gMidpoint;

    int nstep = 4 ;

    for ( int k = 0 ; k< 20 ; k++ ){

        double Is = mySimpson->Integra(nstep);
        double errore = fabs(Is-esatto) ;
        gSimpson.SetPoint(k, mySimpson->GetH(), errore);

        // ...

        double Im = myMidpoint->Integra(nstep);
        errore = fabs(Im-esatto);
        gMidpoint.SetPoint(k, myMidpoint->GetH(), errore );

        // ...

        nstep*=2 ;
    }
```

Costruisco dei TGraph
esterni

Ciclo principale

Deposito l'errore
dentro al TGraph
(possiamo scriverlo
anche in un file per
un utilizzo successivo)

- ❑ Esempio di codice per studiare e grafica l'andamento dell'errore in funzione del passo di integrazione h per i diversi metodi studiati

Costruisco una legenda

```
TLegend leg(0.1,0.7,0.3,0.9);
leg.AddEntry(&gSimpson,"Simpson","lp");
leg.AddEntry(&gMidpoint,"Midpoint","lp");

TCanvas can;
can.cd();

gSimpson.SetTitle("Errore calcolo integrali");
gSimpson.GetXaxis()->SetTitle("Passo h");
gSimpson.GetYaxis()->SetTitle("Errore");
gSimpson.SetMinimum(1E-16);
gSimpson.SetMaximum(1.);
gSimpson.SetLineColor(2);
gSimpson.SetMarkerStyle(20);
gSimpson.Draw("ALP");

gPad-> SetLogy();
gPad-> SetLogx();
gPad->SetGridx();
gPad->SetGridy();

gMidpoint.SetLineColor(3);
gMidpoint.SetMarkerStyle(21);
gMidpoint.Draw("sameLP");

leg.Draw("same");
app.Run();

return 0;
```

Tutti I dettagli del primo
grafico : l'unico con opzione A
(axis) per Draw()

Metto scala log – log e
griglia

Disegno gli altri grafici
con opzione "same"

Stima runtime dell'errore

- E' possibile avere una stima runtime dell'errore che stiamo commettendo integrando con un metodo di quadratura di quelli indicate ? Riutilizziamo il trucchetto della slide precedente

$$\begin{cases} e(h) = I_h - I = k_1 h^2 + k_2 h^4 + \dots \\ e\left(\frac{h}{2}\right) = I_{h/2} - I = k_1 \left(\frac{h}{2}\right)^2 + k_2 \left(\frac{h}{2}\right)^4 + \dots \end{cases}$$

$$I_h - I_{h/2} = \frac{3}{4} k_1 h^2 + o(h^4) \Rightarrow e(h) = k_1 h^2 = \frac{4}{3} (I_h - I_{h/2})$$

- E' possibile stimare quale passo h e' necessario per ottenere una stima dell'integrale con una incertezza fissata ?

$$\begin{aligned} e(h) = k_1 h^2 = \frac{4}{3} (I_h - I_{h/2}) &\Rightarrow k_1 = \frac{4}{3h^2} (I_h - I_{h/2}) \\ \bar{e} = k_1 \bar{h}^2 &\Rightarrow \bar{h} = \sqrt{\frac{\bar{e}}{k_1}} = \sqrt{\frac{\bar{e} 3h^2}{4(I_h - I_{h/2})}} \end{aligned}$$

Algoritmo a precisione fissata

In generale però vorremmo avere un algoritmo che funzioni a precisione fissata non a numero di punti (o passo) fissato.

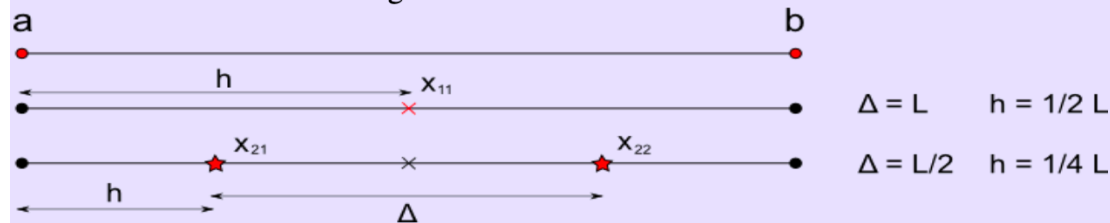
- ❑ Scrivere un algoritmo che stimi l'errore ad ogni passo (vedi slide "stima runtime dell'errore") e raddoppi il numero di intervalli ad ogni passaggio finché non si raggiunge la precision richiesta
- ❑ Il metodo dei trapezoidi si presta ad una versione ottimizzata di questo approccio : ad ogni passaggio aggiungiamo punti al risultato del passo precedente

```
sum0 = (f[a] + f[b]) / 2; I0 = sum0 * (b-a)
```

Al primo passaggio dell'algoritmo suddividiamo l'intervallo in due:

```
sum1 = sum0 + f[x11]; I1 = sum1 * (b-a)/2
```

e così secondo lo schema in figura



```
sum2 = sum1 + f[x21] + f[x22] ; I2 = sum2 * (b-a)/4
```

```
sum3 = sum2 + f[x31] + f[x32] + f[x33] + f[x34]; I3 = sum3 * (b-a)/8
```

Vedi lezione8 dal sito di labTNDs

- ❑ Oppure dalle formule precedenti stimare il passo h necessario per avere un certo errore e fare il calcolo con il passo stimato

Estrapolazione di Richardson

Proviamo a combinare valutazioni dell'integrale con passi diversi per cancellare ordini superiori dell'errore

Trapezoidi

$$\begin{cases} I_h - I = k_1 h^2 + k_2 h^4 + \dots \\ I_{h/2} - I = k_1 \left(\frac{h}{2}\right)^2 + k_2 \left(\frac{h}{2}\right)^4 + \dots \end{cases} \quad \begin{cases} I_h - I = k_1 h^2 + k_2 h^4 + \dots \\ 4I_{h/2} - 4I = k_1 h^2 + 4\frac{1}{16}k_2 h^4 + \dots \end{cases}$$

$$I_h - 4I_{h/2} + 3I = \frac{3}{4}k_2 h^4 + \dots \Rightarrow I = \frac{1}{3}(4I_{h/2} - I_h) + o(h^4)$$

Simpson

$$\begin{cases} I_h - I = k_1 h^4 + k_2 h^6 + \dots \\ I_{h/2} - I = k_1 \left(\frac{h}{2}\right)^4 + k_2 \left(\frac{h}{2}\right)^6 + \dots \end{cases} \quad \begin{cases} I_h - I = k_1 h^4 + k_2 h^6 + \dots \\ 16I_{h/2} - 16I = k_1 h^4 + 16\frac{1}{64}k_2 h^6 + \dots \end{cases}$$

$$I_h - 16I_{h/2} + 15I = \frac{3}{4}k_2 h^6 + \dots \Rightarrow I = \frac{1}{15}(16I_{h/2} - I_h) + o(h^6)$$

Implementazione alternativa : puntatori a funzione, funtori e lambda

```
// this is a function
```

```
double mysinfunction ( double x ) { return sin(x); } ;
```

```
// this is a functor : use it as a function when you need extra parameters. It works like a  
// function as long as you specify the operator()
```

```
class mysinfunctor {
```

```
public :
```

```
    mysinfunctor() { m_para = 1 ;} ;
```

```
    mysinfunctor( double para ) { m_para = para ;} ;
```

```
    double operator() ( double x ) {return m_para * sin(x) ;} ;
```

```
private :
```

```
    double m_para;
```

```
} funct ;
```

Function pointer, not possible to pass external parameters

A functor is basically a class and we use the operator() to mimic a function. Parameters can be passed through constructors or SetParams method

Implementazione alternativa : puntatori a funzione, funtori e lambda

```
// this is simple, but works only with function pointers

double MidpointSimple ( double a, double b, int nsteps, double (*f) (double) ) {

    double integral = 0.5 * ( f(a) + f(b) ) ;
    double delta = (b-a) / nsteps ;

    for ( int i = 1 ; i < nsteps ; i++ ) integral+= f( a+i*delta ) ;

    return integral * delta ;

};

// this is a template function, works with function pointers, functors and lambdas

template<typename Func> double MidpointTemplate ( double a, double b, int nsteps, Func f ) {

    double integral = 0.5 * ( f(a) + f(b) ) ;
    double delta = (b-a) / nsteps ;

    for ( int i = 1 ; i < nsteps ; i++ ) integral+= f( a+i*delta ) ;

    return integral * delta ;

};

// this uses std::function : works with function pointers, functors and lambdas

double MidpointFunction ( double a, double b, int nsteps, std::function<double (double)> f ){

    double integral = 0.5 * ( f(a) + f(b) ) ;
    double delta = (b-a) / nsteps ;

    for ( int i = 1 ; i < nsteps ; i++ ) integral+= f( a+i*delta ) ;

    return integral * delta ;

}
```

Implementazione alternativa : puntatori a funzione, funtori e lambda

```
int main() {

    // MidpointSimple only works with function pointers

    std::cout << MidpointSimple(0,M_PI,10,mysinfunction) << std::endl ;

    // also with lambdas if there's no capture

    auto lambda = [](double x)->double { return sin(x); };

    std::cout << MidpointSimple(0,M_PI,10, lambda ) << endl;
    std::cout << MidpointSimple(0,M_PI,10, [](double x)->double { return sin(x); } ) << endl;
    std::cout << " " << MidpointSimple(0,M_PI,10, sin ) << endl;

    // The other two methods work with function pointers, functors and lambdas

    mysinfunctor m1 ;

    std::cout << MidpointTemplate(0,M_PI,10, mysinfunction ) << endl;
    std::cout << MidpointTemplate(0,M_PI,10, m1 ) << endl;
    std::cout << MidpointTemplate(0,M_PI,10, [&](double x)->double { return sin(x); } ) << endl;

    std::cout << MidpointFunction(0,M_PI,10, mysinfunction ) << endl;
    std::cout << MidpointFunction(0,M_PI,10, mysinfunctor() ) << endl;
    std::cout << MidpointFunction(0,M_PI,10, [](double x)->double { return sin(x); } ) << endl;

    double para = 2;
    mysinfunctor m2(para) ;

    std::cout << MidpointTemplate(0,M_PI,10, m2 ) << endl;
    std::cout << MidpointTemplate(0,M_PI,10, [&](double x)->double { return para*sin(x); } ) << endl;

}
```

Algoritmi di integrazione di equazioni differenziali

Laboratorio Trattamento Numerico dei Dati Sperimentali

L. Carminati
Universita' degli studi di Milano

Equazioni differenziali

- ❑ **DISCLAIMER** : riprendiamo qui solo i punti principali della teoria. Per una trattazione piu' corretta e rigorosa si considerino i corsi di analisi matematica !

Definizione

Un'equazione in cui l'incognita e' una funzione e' detta equazione funzionale.
Un'equazione funzionale in cui compare almeno una derivata della funzione incognita e' detta equazione differenziale.

Definizione

Un'equazione differenziale si dice ordinaria (ODE) se la funzione incognita dipende da una sola variabile, si dice a derivate parziali (PDE) se dipende da piu' variabili.

Definizione

La generica equazione differenziale ordinaria di ordine n ha la seguente forma

$$F(x, y(x), \dots, y^n(x)) = 0$$

dove $y: I \subset \mathbb{R} \rightarrow \mathbb{R}$ e' derivabile n volte in I e $F: A \subset \mathbb{R}^{n+2} \rightarrow \mathbb{R}$

Equazioni differenziali

Definizione

Il massimo ordine di derivazione della funzione incognita e' detto ordine dell'equazione differenziale.

Definizione

L'equazione differenziale e' detta in forma normale se si puo' esplicitare la derivata di ordine massimo, scrivendola nella forma

$$y^n(x) = f(x, y(x), \dots, y^{n-1}(x))$$

con $f : A \subset \mathbb{R}^{n+1} \rightarrow \mathbb{R}$

Definizione

Si dice soluzione dell'equazione differenziale una funzione $y = y(x)$ definita su un intervallo I , derivabile n volte in I , e tale che

$$y^n(x) = f(x, y(x), \dots, y^{n-1}(x)), \quad \forall x \in I$$

Equazioni differenziali

- ❑ In generale per determinare univocamente una soluzione sarà necessario imporre delle condizioni. Le condizioni più comunemente associate a una equazione differenziale ordinaria di ordine n sono le condizioni iniziali

Definizione

Problema di Cauchy o problema ai valori iniziali:

$$\begin{cases} y(x_0) = \bar{y}_0 \\ y'(x_0) = \bar{y}_1 \\ \dots \\ y^{n-1}(x_0) = \bar{y}_{n-1} \\ y^n(x) = f(x, y(x), y'(x), \dots, y^n(x)) \end{cases}$$

Definizione

Si dice soluzione del problema di Cauchy una soluzione definita in un intervallo I avente x_0 come punto interno e verificante le condizioni iniziali .

Equazioni differenziali : esistenza e unicità della soluzione :

□ Esistenza e unicità della soluzione "locale"

- $f: A \subset \mathbb{R}^{n+1} \rightarrow \mathbb{R}$ con A aperto in $\mathbb{R}$
- $(x_0, \vec{y}_0) \in A$
- K compatto contenuto in A
- f e' continua in K
- localmente Lipshitziana rispetto a y , uniformemente rispetto a x
($\exists L > 0$ tc $\forall (x, \vec{y}), (x, \vec{z}) \in K$ risulta $\|f(x, \vec{y}) - f(x, \vec{z})\| < L\|\vec{y} - \vec{z}\|$)

allora

$\exists \delta > 0$ ed esiste una sola funzione $y: [x_0 - \delta, x_0 + \delta] \rightarrow \mathbb{R}$ soluzione del problema di Cauchy

Equazioni differenziali : esistenza e unicità della soluzione :

□ Esistenza e unicità della soluzione "globale"

- $f: [(\alpha, \beta) \times \mathbb{R}^n] \subset \mathbb{R}^{n+1} \rightarrow \mathbb{R}$ con A aperto in $\mathbb{R}$
- f e' continua in $(\alpha, \beta) \times \mathbb{R}^n$
- localmente Lipshitziana rispetto a y , uniformemente rispetto a x
($\exists L > 0$ t.c. $\forall (x, \vec{y}), (x, \vec{z}) \in K$ risulta $\|f(x, \vec{y}) - f(x, \vec{z})\| < L\|\vec{y} - \vec{z}\|$)
- $\|f(t, \vec{y})\| \leq A(t) + B(t)\|\vec{y}\|$ con $A(t)$ e $B(t)$ continue

allora

$\forall t \in (\alpha, \beta), \forall \vec{y}_0 \in \mathbb{R}^n$ esiste un'unica soluzione globale del problema di Cauchy

- Sotto opportune ipotesi sulla funzione f (continuità , lipshitzianita' e sublinearita') possiamo garantire esistenza, unicità e estendibilità della soluzione dell'equazione differenziale (nelle nostre applicazioni non ci porremo particolari problemi)

Equazioni differenziali

- ❑ In generale una equazione differenziale di ordine n si può riscrivere come un Sistema di equazioni differenziali del primo ordine

Data l'equazione $y^n = f(x, y, \dots, y^{n-1})$ introduciamo le funzioni ausiliarie

$$y_0(x) = y(x), y_1(x) = y'(x), y''(x) = y_2(x) \dots y^{n-1}(x) = y_{n-1}$$

Il problema di Cauchy può essere riscritto in questo modo

$$\begin{cases} y_0'(x) = y_1(x) \\ y_1'(x) = y_2(x) \\ \dots \\ y_{n-1}'(x) = f(x, y_0(x), y_1(x), \dots, y_{n-1}(x)) \end{cases} \quad \text{con} \quad \begin{cases} y_0(x_0) = \overline{y_0} \\ y_1(x_0) = \overline{y_1} \\ \dots \\ y_{n-1}(x_0) = \overline{y_{n-1}} \end{cases}$$

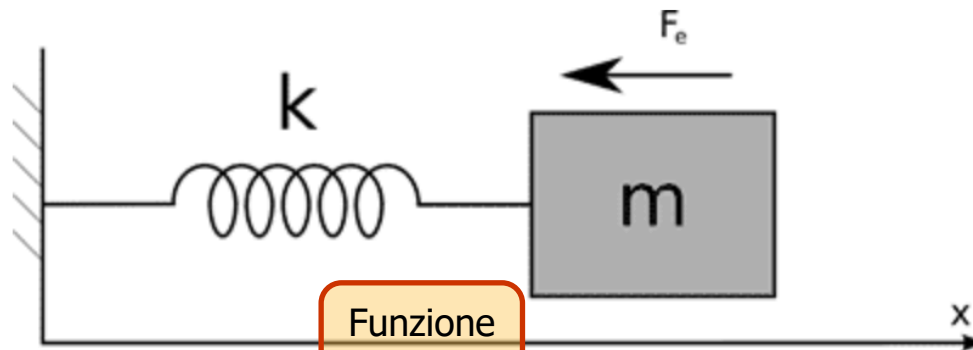
Con $f: A \subset \mathbb{R}^{n+1} \rightarrow \mathbb{R}$

- ❑ Ci possiamo pertanto concentrare sulle tecniche di integrazione di equazioni differenziali di ordine 1

Equazioni differenziali

- Perche' siamo interessati a studiare problemi di questo tipo ? Se sostituiamo y con x e x con t abbiamo magicamente le equazioni della dinamica

$$F = ma = m \frac{d^2}{dt^2} x \Rightarrow \frac{d^2}{dt^2} x = \frac{F}{m}$$



$$\frac{d^2}{dt^2} x = \frac{F}{m} = \frac{kx}{m}$$

Funzione ausiliaria

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = -\frac{k}{m}x \end{cases} \quad \begin{cases} x(t_0) = \bar{x}_0 \\ v(t_0) = \bar{v}_0 \end{cases}$$

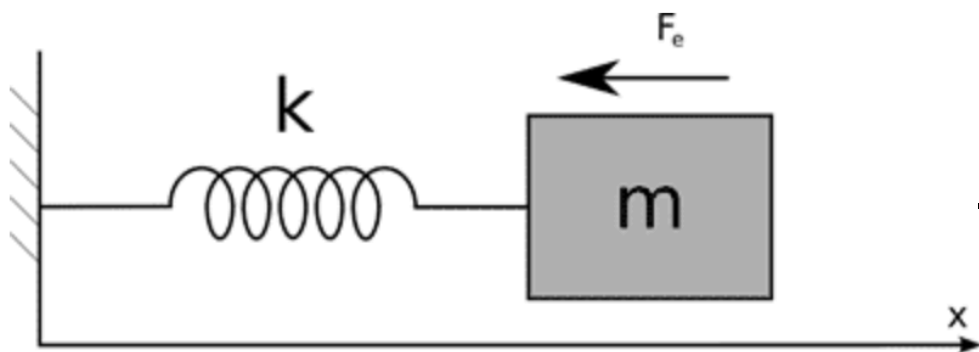
Equazione differenziale di ordine 1

$$\Rightarrow x(t) = A \cos \left(\sqrt{\frac{k}{m}} t + \varphi \right)$$

Equazioni differenziali

- Perché siamo interessati a studiare problemi di questo tipo ? Se sostituiamo y con x e x con t abbiamo magicamente le equazioni della dinamica

$$F = ma = m \frac{d^2}{dt^2} x \Rightarrow \frac{d^2}{dt^2} x = \frac{F}{m}$$



$$\frac{d^2}{dt^2} x = \frac{F}{m} = \frac{k}{m} x - \alpha \frac{dx}{dt} + \sin(\omega t)$$

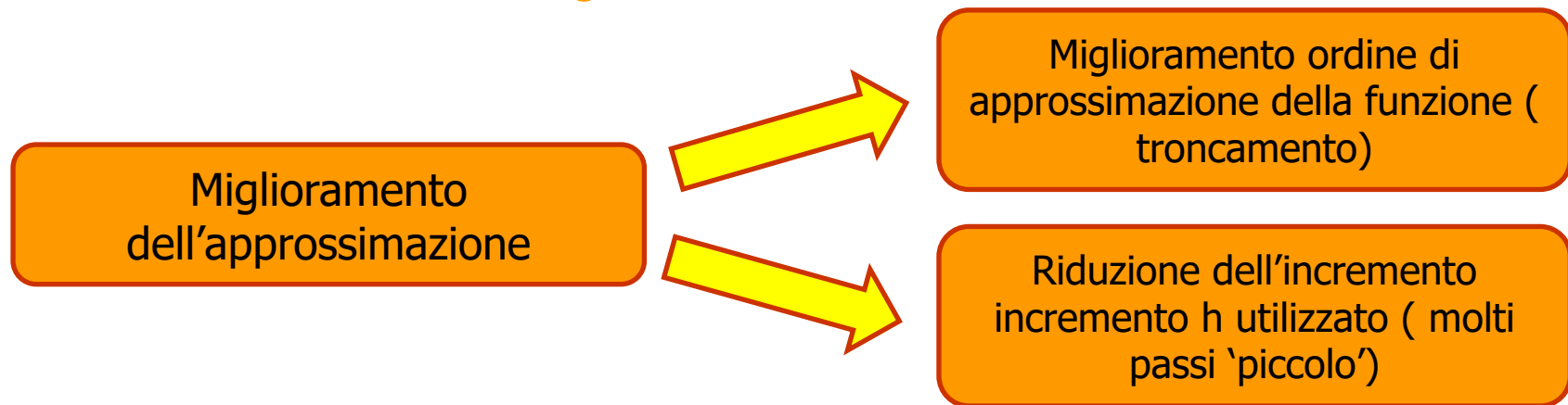
$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = -\omega_0^2 x - \alpha v + \sin(\omega t) \end{cases} \quad \begin{cases} x(t_0) = \overline{x_0} \\ v(t_0) = \overline{v_0} \end{cases}$$

Equazioni differenziali : approccio numerico

- ❑ Proviamo a scrivere degli algoritmi che affrontino il problema dal punto di vista numerico: procediamo per punti !

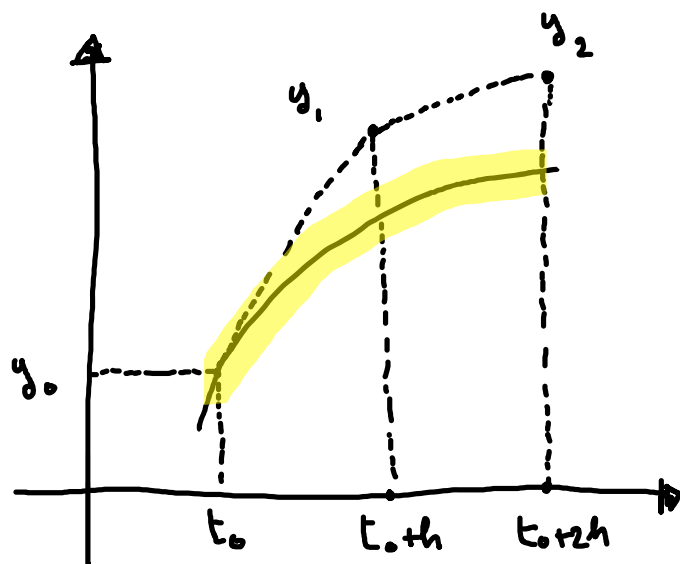
$$y_1 = y(t_0 + h) = y(t_0) + y'(t_0)h + \frac{1}{2}y''(t_0)h^2 + \dots$$

- ❑ Il modo in cui risolviamo il problema e' trovare un set di punti successive $p_i=(t_i, y_i)$ che approssimano la soluzione cercata.
 - ❑ La domanda a cui rispondiamo e' : se il sistema per $t=t_0$ si trova nella posizione y_0 dove si trovera' all'istante generico t ?



Metodo di Eulero

- ❑ Facciamo la cosa piu' semplice che ci viene in mente : tronciamo lo sviluppo di Taylor al primo ordine. La derivate prima ce la fornisce il problema !



$$\begin{cases} y'(t) = f(t, y(t)) \\ y(t_0) = y_0 \end{cases}$$

$$y_1 = y(t_0) + hy'(t_0) + o(h^2)$$

⇓

$$y_n = y(t_n) + hy'(t_n) + o(h^2)$$

- ❑ L'algoritmo e' molto semplice : a partire dal punto noto della soluzione (t_0, y_0) approssimiamo la soluzione con la retta tangente al punto ed estrapoliamo fino a t_0+h . Ripartiamo con lo stesso procedimento a partire dal nuovo punto (t_0+h, y_1)

Caso di sistema fisico :

- ❑ Il caso di un tipico Sistema fisico nel caso monodimensionale (pensiamo al solito oscillatore armónico) e' rappresentato da una equazione di ordine 2 $\Rightarrow$ due equazioni del primo ordine

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv_x}{dt} = -\frac{k}{m}x \end{cases} \quad \begin{cases} x(t_0) = \overline{x_0} \\ v(t_0) = \overline{v_0} \end{cases}$$

- ❑ Dobbiamo quindi pensare di costruire un codice che implementi il semplice algoritmo della slide precedente per le due component (posizione x e velocita' v_x):
- ❑ Ma se il problema fosse piu' complicato (esempio : moto di una particella carica in una regione di campo con presenza di un campo elettrico e magnetico)

Caso di sistema fisico :

- Il moto di una particella carica in una regione di campo con presenza di un campo elettrico e magnetico e' rappresentato da 6 equazioni di ordine 1, 3 per la posizione e 3 per la velocita'

$$\frac{d}{dt} \begin{pmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{pmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & \frac{q}{m} B_z & -\frac{q}{m} B_y \\ 0 & 0 & 0 & -\frac{q}{m} B_z & 0 & \frac{q}{m} B_x \\ 0 & 0 & 0 & \frac{q}{m} B_y & -\frac{q}{m} B_x & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{q}{m} E_x \\ \frac{q}{m} E_y \\ \frac{q}{m} E_z \end{pmatrix}$$

- Meglio pensare al problema in termini vettoriali !

Caso di sistema fisico :

□ Proviamo ad esplicitare tutte le componenti della soluzione con

Usiamo dei vector
per le colonne

$$\begin{pmatrix} x(t+h) \\ y(t+h) \\ z(t+h) \\ v_x(t+h) \\ v_y(t+h) \\ v_z(t+h) \end{pmatrix} = \begin{pmatrix} x(t) \\ y(t) \\ z(t) \\ v_x(t) \\ v_y(t) \\ v_z(t) \end{pmatrix} + h \begin{pmatrix} \frac{d}{dt}x(t) \\ \frac{d}{dt}y(t) \\ \frac{d}{dt}z(t) \\ \frac{d}{dt}v_x(t) \\ \frac{d}{dt}v_y(t) \\ \frac{d}{dt}v_z(t) \end{pmatrix}$$

Qui dobbiamo introdurre
una funzione che accetti
in input il vettore di
stato (x,y,z,v_x,v_y,v_z) e
un istante di tempo t e
ci restituisca il vettore
delle derivate a
quell'istante di tempo

Metodo 'passo' : ci
permette di fare evolvere il
vettore di stato del Sistema
dall'istate t a quello $t+h$

In notazione vettoriale sarebbe
 $\vec{x}(t+h) = \vec{x}(t) + hF(t, \vec{x}(t))$
Diamo un'algebra ai vector !

Algebra dei vettori : definiamo somma e differenza

VectorOperations.h

```
// somma di due vettori : somma componente per componente

template <typename T> std::vector<T> operator+(const std::vector<T> &a, const std::vector<T> &b) {

    assert(a.size() == b.size());
    std::vector<T> result(a.size());

    for (int i = 0; i < static_cast<int>(a.size()); i++) result[i] = a[i] + b[i];

    // Alternativamente si puo' usare l'algoritmo transform della STL
    //
    // std::transform(a.begin(), a.end(), b.begin(), result.begin() , std::plus<T>());

    return result;
}
```

Pesca gli elementi di due vector ed esegue l'operazione specificata
std::transform(
"punto di partenza nel primo vettore",
"punto di termine nel primo vettore",
"punto di partenza nel secondo vettore",
"dove parto per inserire i risultati",
"quale operazione con elementi vettori di input

<http://www.cplusplus.com/reference/algorithm/transform/>

```
// differenza di due vettori componente per componente
// [ preferisco re-implementarlo perche' si fanno meno operazioni rispetto result = a + (-1.*b) ]

template <typename T> std::vector<T> operator-(const std::vector<T> &a, const std::vector<T> &b) {

    assert(a.size() == b.size());
    std::vector<T> result(a.size());

    for (int i = 0; i < static_cast<int>(a.size()); i++) result[i] = a[i] - b[i];

    // Alternativamente si puo' usare l'algoritmo transform della STL
    //
    // std::transform(a.begin(), a.end(), b.begin(), result.begin() , std::minus<T>());

    return result;
}
```

Per utilizzare std::transform
#include <algorithm>

Algebra dei vettori : definiamo addition assignment operator

VectorOperations.h

```
// =====  
// somma ad a un vettore b e il risultato viene messo in a  
// =====  
template <typename T> std::vector<T>& operator+=(std::vector<T>& a, const std::vector<T>& b) {  
    assert(a.size() == b.size());  
  
    for (int i = 0; i < static_cast<int>(a.size()); i++) a[i] += b[i];  
  
    // Alternativamente si puo' usare l'algoritmo transform della STL  
    //  
    // std::transform(a.begin(), a.end(), b.begin(), a.begin() , std::plus<T>());  
  
    return a;  
}
```

Nota : sto sovrascrivendo il
contenuto del vettore a !

Algebra dei vettori : prodotto scalare tra due vettori

VectorOperations.h

```
// =====  
// prodotto scalare tra due vettori  
// =====  
  
template <typename T> T operator*(const std::vector<T> &a, const std::vector<T> &b) {  
  
    assert(a.size() == b.size());  
    T sum = 0 ;  
    for (int i = 0; i < static_cast<int>(a.size()); i++) sum += a[i] * b[i];  
  
    // Alternativamente si puo' usare l'algoritmo inner_product della STL  
    //  
    // sum = std::inner_product(std::begin(a), std::end(a), std::begin(b), 1.0);  
  
    return sum;  
}
```

Per utilizzare std::inner_product
#include <numeric>

Prodotto interno tra gli elementi
std::inner_product(
 "punto di partenza nel primo vettore",
 "punto di termine nel primo vettore",
 "punto di partenza nel secondo vettore",
 "valore iniziale della variabile di accumulazione"

http://www.cplusplus.com/reference/numeric/inner_product/

Algebra dei vettori : definiamo prodotto per uno scalare

VectorOperations.h

```
// =====  
// prodotto tra uno scalare e un vettore  
// =====  
  
template <typename T> std::vector<T> operator*( T c , const std::vector<T> &a) {  
    std::vector<T> result(a.size());  
  
    for (int i = 0; i < static_cast<int>(a.size()); i++) result[i] = c * a[i];  
  
    // Alternativamente si puo' usare l'algoritmo inner product  
    //  
    // std::transform(std::begin(a), std::end(a), std::begin(result), [&c](T x){ return x * c; } );  
  
    return result;  
}
```

```
// =====  
// prodotto tra un vettore e uno scalare  
// =====  
  
template <typename T> std::vector<T> operator*( const std::vector<T> &a , T c) {  
    std::vector<T> result(a.size());  
  
    for (int i = 0; i < static_cast<int>(a.size()); i++) result[i] = c * a[i];  
  
    // oppure il ciclo for puo' essere sostituito da ( ~ stesso numero di operazioni con il  
    // move constructor del vector altrimenti sarebbe meno efficiente )  
    //  
    // result = c * a ;  
  
    // Alternativamente si puo' usare l'algoritmo transform della STL con una lambda function  
    //  
    // std::transform(std::begin(a), std::end(a), std::begin(result), [&c](T x){ return x * c; } );  
  
    return result;  
}
```

Possiamo usare transform passando una lambda function

```
// =====  
// Possiamo usare funzioni generiche che agiscono sui vettori  
// =====  
  
// metodo comodo per stampare il vettore  
  
template <typename T> void Print( const std::vector<T> &v ) {  
    std::cout << "Printing vector" << std::endl;  
    for ( auto it = v.begin() ; it != v.end() ; it++ ) std::cout << *it << " " ;  
    std::cout << std::endl;  
    std::cout << "End of printing vector" << std::endl;  
};
```


Algebra dei vettori : facciamo una prova

```
#include <iostream>
#include "VectorOperations.h"

using namespace std;

int main() {

    vector<double> v1 {2,3,4} ;
    vector<double> v2 {22,33,44};

    cout << "This is v1 " << endl;
    Print (v1) ;
    cout << "This is v2 " << endl;
    Print (v2) ;

    double k = 4.;
    vector<double> v3 = k*v1;
    cout << "===> Multiply v1 by a value k = " << k << endl;
    Print ( v3 );

    cout << "===> Summing v1 and v2 (operator+)" << endl;
    vector<double> v4 = v1+v2 ;
    cout << "This is v1 " << endl;
    Print (v1) ;
    cout << "This is v2 " << endl;
    Print (v2) ;
    cout << "This is v4 " << endl;
    Print (v4) ;

    cout << "===> Summing v1 and v2 (operator+=)" << endl;
    vector<double> v5 = v1+=v2 ;
    cout << "This is v1 (modified !)" << endl;
    Print (v1) ;
    cout << "This is v2 " << endl;
    Print (v2) ;
    cout << "This is v5 " << endl;
    Print (v5) ;

}
```

ProvaAlgebra.cxx

```
This is v1
Printing vector
2 3 4
End of printing vector
This is v2
Printing vector
22 33 44
End of printing vector
===> Multiply v1 by a value k = 4
Printing vector
8 12 16
End of printing vector
===> Summing v1 and v2 (operator+)
This is v1
Printing vector
2 3 4
End of printing vector
This is v2
Printing vector
22 33 44
End of printing vector
This is v4
Printing vector
24 36 48
End of printing vector
===> Summing v1 and v2 (operator+=)
This is v1 (modified !)
Printing vector
24 36 48
End of printing vector
This is v2
Printing vector
22 33 44
End of printing vector
This is v5
Printing vector
24 36 48
End of printing vector
```

Possibile struttura delle classi

FunzioneVettorialeBase

```
virtual vector<double> Eval(double t, const  
vector<double> & x) const = 0 ;
```

Oscillatore, pendolo, ...

```
vector<double> Eval(double t, const  
vector<double> & x) const { ... } ;
```

EquazioneDifferenzialeBase

```
virtual vector<double> Passo (double t, const vector<double> & x,  
double h, FunzioneVettorialeBase *f)=0;
```

Eulero, Runge-Kutta, ..

```
virtual vector<double> Passo (double t, const vector<double> & x,  
double h, FunzioneVettorialeBase *f) { ... } ;
```

Rappresentiamo il problema fisico:

- ❑ Un problema fisico reale e' definito dalle sue derivate : codifichiamo un generico problema come una `FunzioneVettorialeBase` (astratta). Ogni sistema concreto ereditera' da `FunzioneVettorialeBase`

```
// =====  
// classe astratta, restituisce la derivata da valutata nel punto x  
// =====  
  
class FunzioneVettorialeBase {  
public:  
    virtual vector<double> Eval(double t, const vector<double> & x) const = 0;  
};  
  
// caso fisico concreto, oscillatore armonico  
  
class OscillatoreArmonico : public FunzioneVettorialeBase {  
public:  
    OscillatoreArmonico(double omega0) { m_omega0 = omega0; } ;  
  
    virtual vector<double> Eval(double t, const vector<double> & x) const {  
  
};  
  
private:  
    double m_omega0;  
};
```

Metodo virtuale puro : restituisce il vettore delle derivate calcolate rispetto ad un vettore di stato x

Dobbiamo implementare qui la derivata: restituire un vector le cui componenti rappresentano le derivate all'istante t. Qui codifichiamo il contenuto fisico del problema

Rappresentiamo la parte algoritmica

- ❑ Soluzione piu' semplice : proviamo ad approssimare la funzione utilizzando dei polinomi di grado n

```
// =====  
// classe astratta per un integratore di equazioni differenziali  
// =====  
  
class EquazioneDifferenzialeBase {  
public:  
    virtual vector<double> Passo(double t, const vector<double>& x, double h, FunzioneVettorialeBase *f) const =0;  
};  
  
// integratore concreto, metodo di Eulero  
  
class Eulero : public EquazioneDifferenzialeBase {  
public:  
    virtual vector<double> Passo(double t, const vector<double> & x, double h, FunzioneVettorialeBase *f) const {  
  
    };  
};
```

Sfruttiamo di nuovo il polimorfismo : il metodo agisce su una FunzioneBase* . Il metodo Eval all'interno cambiera' a seconda del puntatore che entra

Dobbiamo codificare qui come funziona il metodo di Eulero : utilizzando l'algebra dei vettori il metodo divente indipendente dalle dimensioni del problema !

Approssimazione polinomiale : teoremi fondamentali

```
#include "EquazioniDifferenziali.h"

#include <iostream>
#include <string>

#include "TApplication.h"
#include "TAxis.h"
#include "TCanvas.h"
#include "TGraph.h"

int main (int argc, char** argv ) {

    if ( argc!=2 ) {
        std::cerr << "Usage: " << argv[0] << " <stepsize>" << std::endl;
        return -1;
    }

    TApplication myApp("myApp",0,0);

    Eulero myEuler;

    OscillatoreArmonico *osc = new OscillatoreArmonico(1.);

    double tmax = 70.;
    double h = atof(argv[1]);
    vector<double> x {0.,1.} ;
    double t = 0.;

    TGraph *myGraph = new TGraph();
    int nstep = int(tmax/h+0.5);

    for (int step=0; step<nstep; step++) {
        myGraph->SetPoint(step,t,x[0]);
        x = myEuler.Passo(t,x,h,osc);
        t = t+h;
    }

    TCanvas *c = new TCanvas();
    c->cd();
    string title = "Oscillatore armonico (Eulero h = " + std::to_string(h) + ")" ;
    myGraph->SetTitle(title.c_str());
    myGraph->GetXaxis()->SetTitle("Tempo [s]");
    myGraph->GetYaxis()->SetTitle("Posizione x [m]");
    myGraph->Draw("AL");

    myApp.Run();
}
```

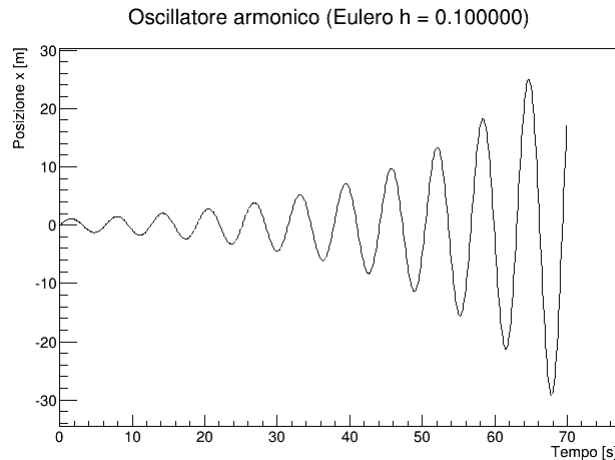
Algoritmo di integrazione

Ciclo base del codice : la costruzione della soluzione avviene per chiamate successive del metodo Passo

Problema fisico : eredita da FunzioneVettorialeBase

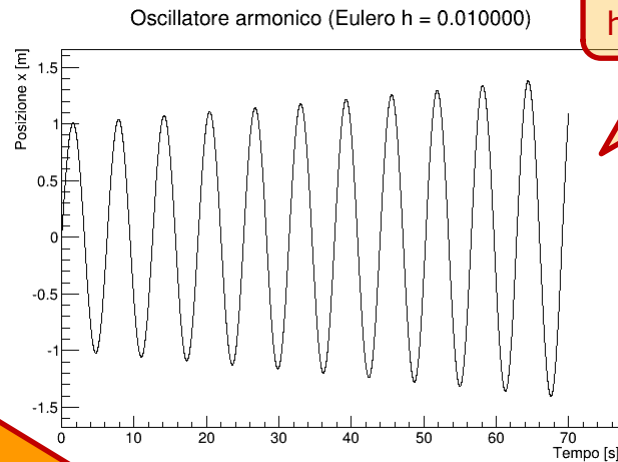
Polimorfismo : entra un OscillatoreArmonico *

Integrazione di equazioni differenziali con il metodo di Eulero



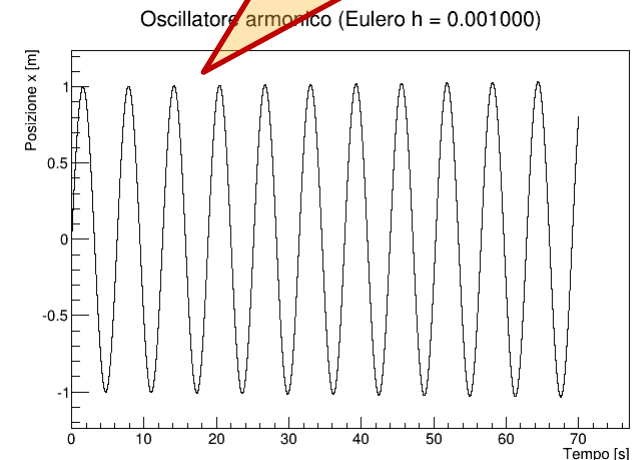
Integriamo l'oscillatore armonico da 0 a 70 s con diversi passi

Passo di integrazione
 $h=0.1$: 700 integrazioni



Passo di integrazione
 $h=0.01$: 7000 integrazioni

Passo di integrazione
 $h=0.001$: 70000 integrazioni



Integrazione dell'equazione differenziale con passo di integrazione che diminuisce: interessante studiare come va l'errore tra la nostra soluzione e la soluzione esatta al variare del passo h per un tempo fissato (diciamo $t=70$ s)

Metodo di Runge-Kutta di ordine 2

- ❑ Possiamo migliorare il metodo di integrazione cercando di migliorare l'ordine di troncamento utilizzato per il calcolo ?

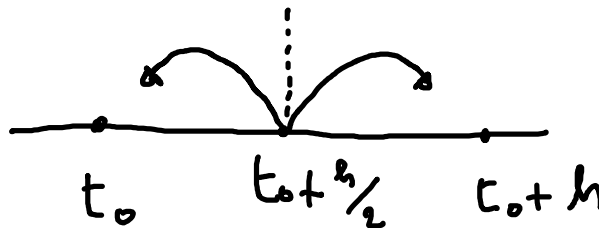
$$\begin{cases} y'(t) = f(t, y(t)) \\ y(t_0) = t_0 \end{cases}$$



$$y_1 = y(t_0) + hy'(t_0) + \frac{1}{2}h^2y''(t_0) + o(h^3)$$

La derivate seconda purtroppo non e' nota.
Dobbiamo calcolarla o trovare un modo di
cancellare questo termine di errore

- ❑ Proviamo a questo punto a scrivere lo sviluppo di Taylor della ipotetica soluzione a partire dal punto di mezzo, sviluppando in avanti e all'indietro



Metodo di Runge-Kutta di ordine 2

$$y(t_0 + h) = y\left(t_0 + \frac{h}{2}\right) + \frac{h}{2}y'\left(t_0 + \frac{h}{2}\right) + \frac{h^2}{8}y''\left(t_0 + \frac{h}{2}\right) + o(h^3)$$

$$y(t_0) = y\left(t_0 + \frac{h}{2}\right) - \frac{h}{2}y'\left(t_0 + \frac{h}{2}\right) + \frac{h^2}{8}y''\left(t_0 + \frac{h}{2}\right) + o(h^3)$$


$$y(t_0 + h) - y(t_0) = hy'\left(t_0 + \frac{h}{2}\right) + o(h^3)$$

Questa non e' nota
dal problema

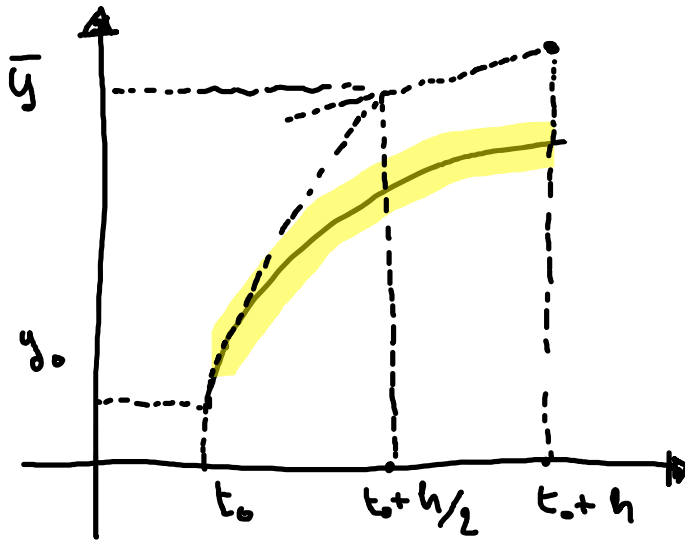
Possiamo applicare Eulero
(errore $\sim h^2$), questo termine
viene poi moltiplicato per h

$$y'\left(t_0 + \frac{h}{2}\right) = f\left(t_0 + \frac{h}{2}, y\left(t_0 + \frac{h}{2}\right)\right) \cong f\left(t_0 + \frac{h}{2}, y_0 + \frac{h}{2}f(t_0, y_0)\right)$$


$$y(t_0 + h) = y(t_0) + hf\left(t_0 + \frac{h}{2}, y_0 + \frac{h}{2}f(t_0, y_0)\right) + O(h^3)$$

Metodo di Runge-Kutta di ordine 2

- Formalizziamo il metodo che abbiamo appena realizzato (metodo di Runge di ordine 2) con errore locale di ordine h^3



$$\begin{aligned} K_1 &= f(t_n, y_n) \\ K_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} K_1\right) \\ y_{n+1} &= y_n + h K_2 \end{aligned}$$

- L'algoritmo e' molto semplice : a partire dal punto noto della soluzione (t_0, y_0) approssimiamo la soluzione con la retta tangente al punto ed estrapoliamo fino a $t_0 + h$. Ripartiamo con lo stesso procedimento a partire dal nuovo punto $(t_0 + h, y_1)$

Metodo di Runge-Kutta di ordine 4

- Utilizzando sviluppi opportuni della funzione in punti vicini e' possibile costruire un algoritmo di integrazione di equazioni differenziali con errore locale di ordine h^5

$$K_1 = f(t_n, y_n)$$

$$K_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} K_1\right)$$

$$K_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} K_2\right)$$

$$K_4 = f(t_n + h, y_n + h K_3)$$

$$y_{n+1} = y_n + \frac{h}{6} (K_1 + 2K_2 + 2K_3 + K_4) + o(h^5)$$

Stima runtime dell'errore

- Integrando una equazione differenziale di cui non conosciamo la soluzione esatta come possiamo stimare l'errore che stiamo commettendo nella stima della soluzione ad un istante t (dopo N passi) ? Possiamo utilizzare un trucchetto come quello già' usato per le formule di quadrature numerica

$$\begin{cases} y_N - y = kh^4 \quad (= \varepsilon) \\ y_{2N} - y = k \left(\frac{h}{2}\right)^4 \end{cases}$$

$$\Delta_N = y_N - y_{2N} = \frac{15}{16}kh^4 \Rightarrow \varepsilon = kh^4 = \Delta_N \frac{16}{15}$$

- Come stimare il passo h necessario per ottenere una precisione $\bar{\varepsilon}$?